

CROWD-SOURCED FAQ GENERATION PLATFORM

AI Team — Complete Strategy, Architecture & Execution Roadmap

Version 1.0 | Sprint Duration: 1 Week | Team: 7 Members

Field	Details
Project Objective	Minimize time delay between a user raising a query and receiving an answer
AI Stack	MiniMax-2 API · Embeddings · Semantic Vector Search · Prompt Engineering
Frontend	React · TanStack Query · TanStack Router
Backend	NestJS · Express · MongoDB · Mongoose
Team Size (AI)	7 members
Sprint Duration	~7 days · ~4 hrs/day per person
Priority 0 Features	Semantic Search, FAQ Suggestions, Classification, Moderation, FAQ Generation, Tags

01 Complete AI Deliverables — What We Are Building

Below is every intelligent component the AI team owns. Each deliverable maps directly to a node in the user flow where the system must make an automated, intelligent decision.

D-01 Semantic FAQ Search [P0] Owner: Gurnoor

Find existing FAQs that mean the same as the user's query even when the wording is completely different. This is the highest-priority feature and directly fulfils the project objective.

Outputs:

- Embedding generation for all FAQs and incoming queries
- Cosine similarity / dot-product scoring engine
- Configurable confidence threshold (recommend starting at 0.78)
- Top-N FAQ retrieval with ranked results
- REST endpoint: POST /ai/search → { results[], confidence }

D-02 Similar FAQ Suggestions (Live) [P0] Owner: Gurnoor

While the user is still typing on the Raise Query page, surface FAQs in real time to encourage self-service and prevent duplicate queries.

Outputs:

- Debounced semantic search triggered on keystroke (300ms delay)
- Returns top-3 suggestions with similarity scores
- REST endpoint: POST /ai/suggest → { suggestions[] }
- Threshold lower than D-01 (recommend 0.65) to surface more options

D-03 Query Classification [P0] Owner: Vishal

Automatically decide whether a submitted query is Generic (answerable by community) or Personal (requires private/admin handling). This gates the routing logic.

Outputs:

- LLM prompt via MiniMax-2 with few-shot examples
- Output: { type: 'generic' | 'personal', confidence: float }
- Vishal defines routing rules based on classification output
- Edge-case handling for ambiguous queries (default to 'generic' if confidence < 0.6)

D-04 Query Moderation [P0] Owner: Vishal

Screen every submitted query for profanity, spam, abuse, and toxic language before it enters the community system.

Outputs:

- LLM-based content safety check via MiniMax-2
- Output: { safe: true | false, reason?: string }
- Samarpit exposes moderateQuery() wrapper for Vishal to consume
- Hard block on 'safe: false' — query does not proceed

D-05 FAQ Generation from Community Answers [P0] Owner: Negha

After a question has accumulated community answers, use AI to synthesise a clean, reusable FAQ entry that enters the knowledge base.

Outputs:

- Input: question + array of community answers
- Output: { faqQuestion, faqAnswer, tags[], quality_score }
- Structured prompt with JSON-mode output
- Negha owns FAQ formatting templates and quality thresholds

D-06 Automatic Tag Generation [P0] Owner: Negha

Generate 3–5 relevant, normalised tags for every FAQ to power better search and categorisation.

Outputs:

- Tags extracted from FAQ content via MiniMax-2
- Normalised to lowercase, deduplicated against existing tag vocabulary
- Output: { tags: string[] }
- Tags stored on FAQ document in MongoDB

D-07 FAQ Quality Review [P1] Owner: Negha

Before an AI-generated FAQ is published, automatically evaluate its clarity, completeness, duplication risk, and relevance.

Outputs:

- Quality dimensions: clarity, completeness, duplication check, relevance score
- Output: { approved: bool, score: float, issues: string[] }
- Triggers automatically after D-05 (FAQ Generation)
- Human review required only if score < 0.70

D-08 Feedback Collection & Storage [P1] Owner: Riddhima

Collect Helpful / Not Helpful signals on every served FAQ. Store structured feedback for future model improvement cycles.

Outputs:

- Feedback events: { faqId, userId, signal: 'helpful'|'not_helpful', timestamp }
- Riddhima defines feedback schema and lifecycle rules
- Backend team (Lakshit/Remy/Madhav) builds the API endpoint
- Aggregated feedback score exposed on FAQ document

02

Resource Division — Logic & Rationale

The division is not random. It follows a single design principle:

Core Division Principle

- Group responsibilities by cognitive domain, not by deliverable count.
- Every person owns one conceptual layer end-to-end rather than sharing layers across people.
- No deliverable crosses two AI team members — exactly one person is accountable for each output.
- Shared infrastructure (MiniMax API wrapper, prompt library) is owned by one person (Samarpit) and consumed by all others.

Why Each Person Got Their Role

Person	Logic Behind Assignment
Gurnoor (AI Lead)	Owns the Search layer — the single highest-priority feature. As AI Lead, all cross-team coordination flows through this role. Semantic search requires deepest ML intuition.
Samarpit	Owns the AI Service Layer — acts as internal platform engineer. Every other AI team member calls functions Samarpit builds. This removes API integration complexity from everyone else.
Vishal	Owns the Routing & Safety layer — classification and moderation are logically connected (classify → moderate → route). Grouping them prevents interface mismatches.
Negha	Owns the Knowledge Creation layer — FAQ generation, tagging, and quality review are a pipeline. One person owning all three ensures consistent FAQ format and quality bar.
Riddhima (Lead)	Owns the Product & Lifecycle layer — feedback, approval workflows, and FAQ states require product judgment more than engineering. Correct fit for the project lead.
Gunjan	Owns Documentation & Architecture Diagrams — outputs that require understanding the whole system. Best suited after being embedded in the AI architecture discussions.
Rishi	Owns Testing & Evaluation — dataset creation, edge-case testing, and evaluation reports require independence from the builders to produce honest quality metrics.

03

Individual Tasks, Skills & Responsibilities

Gurnoor

AI Lead · Semantic Search Architect

OWNS

- Semantic FAQ Search (D-01) — core engine
- Live FAQ Suggestions (D-02)
- Embedding model selection & evaluation
- Confidence threshold definition and tuning
- AI ↔ Backend API contract coordination
- Final AI integration testing across all modules

SKILLS TO LEARN

- Vector embeddings: what they are, how cosine similarity works
- MiniMax-2 embeddings API (or evaluate alternatives: text-embedding-ada-002)
- Vector search in MongoDB (Atlas Vector Search) or in-memory with cosine computation
- Choosing and tuning similarity thresholds (precision vs recall tradeoff)
- REST API design for search endpoints consumed by frontend
- Debounce strategy for live-suggestion endpoint
- Integration testing patterns for AI APIs in NestJS

TASKS

1. Select and benchmark embedding model — run sample FAQ corpus through candidate models
2. Design vector index schema in MongoDB (or decide in-memory for MVP)
3. Implement POST /ai/search with top-N retrieval and confidence scoring
4. Implement POST /ai/suggest with lower threshold for live suggestions
5. Define similarity thresholds in a config file (easy to tune)
6. Write API contract doc for backend team (request shape, response shape, error codes)
7. Coordinate integration testing with Samarpit's service layer
8. Own final end-to-end AI integration test before demo

Samarpit

MiniMax Integration · AI Service Layer

OWNS

- MiniMax-2 API wrapper (single authenticated client)
- Prompt library and prompt versioning
- Structured JSON output parsing and validation
- Reusable function library consumed by all AI team members

SKILLS TO LEARN

- MiniMax-2 API authentication, rate limits, and cost model
- Prompt engineering: system prompts, few-shot examples, chain-of-thought
- Enforcing structured/JSON output from LLMs (JSON mode, output parsers)
- Error handling and retry logic for external API calls in NestJS
- TypeScript interfaces for all AI function inputs and outputs
- Environment variable management for API keys in NestJS

TASKS

9. Build MiniMax API client with retry, timeout, and error handling
10. Implement `classifyQuery(query: string) → { type, confidence }`
11. Implement `moderateQuery(query: string) → { safe, reason? }`
12. Implement `generateFAQ(question, answers[]) → { faqQuestion, faqAnswer, tags, quality_score }`
13. Implement `generateTags(content: string) → { tags[] }`
14. Implement `reviewFAQQuality(faq) → { approved, score, issues[] }`
15. Write prompt documentation for every function in the prompt library
16. Provide TypeScript interfaces so Vishal and Negha can consume functions type-safely

Vishal

Query Classification · Moderation · Routing

OWNS

- Query Classification module (D-03)
- Query Moderation module (D-04)
- Routing logic: Generic → Community, Personal → Admin/DM

SKILLS TO LEARN

- How LLM classification works vs traditional classifiers
- Few-shot prompting for classification accuracy
- Calling Samarpit's `classifyQuery()` and `moderateQuery()` from NestJS service
- Designing routing rules as a decision tree in code
- NestJS service and controller patterns
- How to handle classification edge cases (low confidence, ambiguous queries)

TASKS

17. Consume `classifyQuery()` from Samarpit's service layer
18. Consume `moderateQuery()` from Samarpit's service layer
19. Implement routing service: if `safe=false` → reject; if `type=personal` → route to admin; else → community queue
20. Write routing rules in a config object so Riddhima can adjust without code changes
21. Handle low-confidence edge cases: default=generic if confidence < 0.6
22. Unit test classification + moderation with at least 20 labelled examples
23. Coordinate with backend team to connect routing output to Query API

Negha

FAQ Generation · Tagging · Quality Review

OWNS

- FAQ Generation from community answers (D-05)
- Tag Generation (D-06)
- FAQ Quality Review (D-07)
- FAQ formatting templates and output structure

SKILLS TO LEARN

- Prompt engineering for summarisation and synthesis tasks
- How to design FAQ templates (question framing, answer length norms)
- Calling `generateFAQ()`, `generateTags()`, `reviewFAQQuality()` from Samarpit's library

- Understanding FAQ quality dimensions: clarity, completeness, duplication, relevance
- MongoDB document design for FAQ collection (question, answer, tags, status, quality_score)
- When to trigger quality review automatically vs when to escalate to human

TASKS

24. Define FAQ output schema: { faqQuestion, faqAnswer, tags[], quality_score, status }
25. Consume generateFAQ() — validate output against schema, handle malformed responses
26. Consume generateTags() — normalise tags (lowercase, deduplicate against existing tags)
27. Consume reviewFAQQuality() — auto-approve if score ≥ 0.70 , else flag for human review
28. Implement FAQ Generation trigger: fires when a question has ≥ 2 community answers
29. Write FAQ formatting style guide (1 page) so generated FAQs are consistent
30. Coordinate with Riddhima on approval workflow handoff after quality review

Riddhima

Project Lead · Product Flow · FAQ Lifecycle

OWNS

- Feedback Collection system design (D-08)
- FAQ lifecycle state machine: Draft $\rightarrow$ Pending Review $\rightarrow$ Approved $\rightarrow$ Published $\rightarrow$ Archived
- Approval workflow: who can approve, what triggers each state change
- Knowledge Management Flow — how answered queries strengthen the FAQ database

SKILLS TO LEARN

- State machine design for document lifecycle in MongoDB
- Feedback schema design: what to capture, how to aggregate
- Product thinking: how to define done for each AI deliverable
- API design for feedback endpoint (consumed by frontend team)
- How to write acceptance criteria for AI features (output-focused, not implementation-focused)

TASKS

31. Define FAQ state machine with all valid transitions and who triggers them
32. Define feedback schema: { faqlId, userId, signal, timestamp, sessionId }
33. Write acceptance criteria for all 8 AI deliverables (Rishi uses these for testing)
34. Coordinate with backend team: define Feedback API endpoint shape
35. Define what 'knowledge base updated' means operationally (which DB write, who triggers it)
36. Own the FAQ approval workflow: define when AI auto-approves vs human review needed
37. Scope and prioritisation calls: cut anything that risks the P0 deliverables

Gunjan

AI Documentation · Architecture Diagrams

OWNS

- Complete AI Architecture documentation
- Flow diagrams for all AI-involved user journeys
- Prompt documentation for every function in Samarpit's library
- API documentation for all AI endpoints
- Presentation assets for demo day

SKILLS TO LEARN

- How to read and document system architecture diagrams (draw.io or Figma)

- How to write API docs (request, response, error codes, example payloads)
- How to document prompts: name, purpose, input variables, expected output, edge cases
- Mermaid.js or similar for inline diagrams in markdown

TASKS

38. Document full AI architecture: components, data flows, technology choices
39. Create user flow diagram: from query input to FAQ served (all branches)
40. Document every prompt in Samarpit's library with inputs, outputs, and example calls
41. Write API reference for all AI endpoints (Gurnoor's search + Samarpit's functions)
42. Create one-page AI team contribution summary for final presentation
43. Review documentation completeness with each team member mid-sprint

Rishi

AI Testing · Evaluation · Dataset Creation

OWNS

- FAQ test dataset (minimum 50 FAQ entries)
- Query test dataset (minimum 40 query variations)
- Semantic search evaluation (Precision@K, MRR)
- Classification and moderation evaluation (accuracy, false positive rate)
- Edge case testing across all AI deliverables
- Evaluation reports for demo day

SKILLS TO LEARN

- Evaluation metrics for information retrieval: Precision@K, Recall@K, MRR
- Classification evaluation metrics: accuracy, precision, recall, F1, false positive rate
- How to write test datasets for NLP tasks (diverse examples, edge cases, adversarial inputs)
- How to structure evaluation reports for non-technical stakeholders
- Basic scripting in Node.js to automate evaluation runs against live endpoints

TASKS

44. Build FAQ dataset: 50 real-looking FAQs across 5 categories (account, access, grades, fees, general)
45. Build query dataset: 40 queries — 10 with clear matches, 10 with semantic matches, 10 edge cases, 10 adversarial
46. Run semantic search evaluation: measure Precision@3, MRR against known matches
47. Build classification test set: 20 generic, 20 personal, 5 ambiguous — measure accuracy
48. Build moderation test set: 15 safe, 10 toxic, 5 edge cases — measure false positive rate
49. Evaluate FAQ generation output quality against Riddhima's acceptance criteria
50. Write final evaluation report: metrics per deliverable, pass/fail vs targets

04

Full AI Architecture

Architecture Layers

Layer	Components & Technologies
Client Layer	React frontend · TanStack Query for API state · TanStack Router for page routing
API Gateway	NestJS controllers expose AI endpoints · Express middleware for auth/rate-limit
AI Orchestration	Gurnoor's Search Service · Vishal's Classification+Moderation Service · Negha's FAQ Generation Service
AI Service Layer	Samarpit's MiniMax wrapper · Prompt library · Structured output parser · TypeScript interfaces
MiniMax-2 API	LLM completions for classification, moderation, FAQ generation, tagging, quality review
Embedding Engine	Vector embeddings (MiniMax embeddings or text-embedding-ada-002) · Cosine similarity engine
Vector Store	MongoDB with vector index (Atlas Vector Search) OR in-memory JS for MVP
Database	MongoDB · Mongoose · Collections: faqs, queries, answers, feedback, embeddings
Feedback Layer	Riddhima's feedback schema · Aggregated helpful/not-helpful signals stored on FAQ documents

Complete User Flow — With AI Decision Points

FAQ Page Flow (Returning User — FAQ Found)

- 1. User opens FAQ page → types search query
- 2. POST /ai/search called with query text
- 3. [AI: Gurnoor] Query embedded → cosine similarity against all FAQ embeddings
- 4. If top match confidence ≥ 0.78 → FAQ returned to UI
- 5. User reads FAQ → clicks Helpful or Not Helpful
- 6. Feedback stored → [Riddhima] feedback aggregated on FAQ document

Raise Query Flow (No FAQ Found)

- 1. User navigates to Raise Query page
- 2. User types query → POST /ai/suggest fires (debounced, 300ms)
- 3. [AI: Gurnoor] Top-3 suggestions returned if confidence ≥ 0.65
- 4. User dismisses suggestions and submits query
- 5. [AI: Vishal] POST /ai/classify → { type: 'generic' | 'personal' }
- 6. [AI: Vishal] POST /ai/moderate → { safe: true | false }

- 7. If safe=false → query rejected with reason shown to user
- 8. If type=personal → routed to admin/DM handling (out of AI scope)
- 9. If type=generic AND safe=true → query enters community discussion queue

Community → FAQ Generation Flow

1. Community members submit answers on Answer Query page
2. Trigger condition: query has ≥ 2 upvoted community answers
3. [AI: Negha] POST /ai/generate-faq with question + answers[]
4. [AI: Samarpit's generateFAQ()] → { faqQuestion, faqAnswer, tags[], quality_score }
5. [AI: Negha] POST /ai/review-faq → { approved, score, issues[] }
6. If score ≥ 0.70 → FAQ auto-approved, status set to 'Pending Review'
7. If score < 0.70 → flagged for human review by Riddhima's workflow
8. [Riddhima] FAQ approved → status → 'Published' → added to FAQ embedding index
9. [AI: Gurnoor] New FAQ embedding generated and indexed → knowledge base grows

AI Endpoints Reference

Endpoint	Owner · Input → Output
POST /ai/search	Gurnoor · { query } → { results[{ faqlId, question, answer, confidence }] }
POST /ai/suggest	Gurnoor · { query } → { suggestions[{ faqlId, question, confidence }] }
POST /ai/classify	Vishal → Samarpit · { query } → { type: 'generic' 'personal', confidence }
POST /ai/moderate	Vishal → Samarpit · { query } → { safe: bool, reason?: string }
POST /ai/generate-faq	Negha → Samarpit · { question, answers[] } → { faqQuestion, faqAnswer, tags[], quality_score }
POST /ai/generate-tags	Negha → Samarpit · { content } → { tags[] }
POST /ai/review-faq	Negha → Samarpit · { faq } → { approved: bool, score, issues[] }
POST /feedback	Riddhima (design) · { faqlId, signal } → { ok: bool }

05

Execution Phases & 7-Day Roadmap

The sprint is divided into 4 phases. Each phase builds on the last. No phase starts until the previous phase's gates are met.

Phase 1: Foundation & Setup Day 1–2

Goal: Everyone has a working dev environment, all APIs are authenticated, data schemas are agreed.

Tasks

- Samarpit: MiniMax API client up, all 4 functions stubbed with TypeScript interfaces
- Gurnoor: Embedding model selected, benchmark run on 20 sample FAQs, vector index decision made
- Vishal: Routing decision tree documented, classification + moderation flow diagrams done
- Negha: FAQ output schema defined, generation trigger rules documented
- Riddhima: FAQ state machine documented, feedback schema agreed with backend team
- Rishi: Test datasets started (20 FAQs, 15 queries), evaluation metric definitions written
- Gunjan: Architecture diagram V1 drafted, API contract template shared

Deliverables

- MiniMax API wrapper live and callable by all team members
- Embedding model selected with benchmark results
- All schemas and interfaces defined (TypeScript + MongoDB)
- Phase 1 gate: Samarpit's `classifyQuery()` returns correct output on 5 manual test cases

Phase 2: Core AI Build Day 3–5

Goal: All P0 deliverables implemented and individually testable in isolation.

Tasks

- Samarpit: All 4 functions fully implemented with real MiniMax calls and error handling
- Gurnoor: `/ai/search` endpoint live with real embeddings and cosine scoring
- Gurnoor: `/ai/suggest` endpoint live with lower threshold and debounce
- Vishal: `/ai/classify` and `/ai/moderate` endpoints integrated with Samarpit's functions
- Vishal: Routing service implemented and connected to Query API
- Negha: `/ai/generate-faq` endpoint live — accepts question + answers, returns structured FAQ
- Negha: `/ai/generate-tags` and `/ai/review-faq` endpoints live
- Rishi: Runs evaluation on each endpoint as it goes live — flags failures immediately
- Gunjan: API reference docs written for each completed endpoint

Deliverables

- All 6 P0 AI endpoints live and individually callable via Postman/curl
- Semantic search $\text{Precision@3} \geq 0.75$ on Rishi's test dataset
- Classification accuracy ≥ 0.85 on Rishi's labelled test set
- Moderation false positive rate $\leq 10\%$ on Rishi's edge case set
- Phase 2 gate: Full happy-path flow works end-to-end without frontend

Phase 3: Integration & P1 Features Day 5–6

Goal: All AI endpoints integrated with frontend and backend. P1 features built.

Tasks

- Gurnoor: Integration testing with frontend FAQ search page and Raise Query page
- Gurnoor: Coordinate with backend team — verify all AI endpoints wired correctly in NestJS
- Negha + Riddhima: FAQ Generation trigger wired to Answer Query page (fires after 2 answers)
- Negha: FAQ quality review auto-approval flow live
- Riddhima: Feedback endpoint integrated with frontend (Helpful / Not Helpful buttons)
- Vishal: End-to-end routing flow tested: submit query → classify → moderate → route
- Rishi: Full end-to-end evaluation on integrated system — measure against all targets
- Gunjan: Architecture diagram V2 updated to reflect actual implementation

Deliverables

- Complete user journey works end-to-end: query in → FAQ served (if exists)
- Complete user journey works end-to-end: query in → community → FAQ generated → published
- Feedback captured and stored correctly
- All P1 features live
- Phase 3 gate: Demo scenario runs without errors

Phase 4: Polish, Testing & Demo Prep Day 7

Goal: Fix bugs, hit evaluation targets, prepare demo assets.

Tasks

- Rishi: Final evaluation report written — all metrics vs targets
- Rishi + Gurnoor: Threshold tuning based on evaluation results
- Gunjan: Final architecture diagram, prompt documentation, and presentation slides
- All: Bug fixes only — no new features
- Riddhima: Demo scenario rehearsal — walk through full flow live
- Riddhima: Scope call — explicitly confirm what is and is not demoed

Deliverables

- Final evaluation report with all metrics
- Complete documentation package: architecture, API reference, prompt docs
- Demo-ready system: happy path works consistently
- Presentation deck

067-Day Sprint Roadmap — Who Does What When

Person	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Gurnoor	Model select + benchmark	Vector index design	Build /ai/search	Build /ai/suggest	Integration test	Frontend wiring	Tuning + demo
Samarpit	API client + stubs	classifyQuery() + moderateQuery()	generateFAQ() + generateTags()	reviewFAQQuality() + error handling	Support integration	Bug fixes	Prompt docs
Vishal	Flow diagram + routing rules	Consume classify + moderate	/ai/classify live	/ai/moderate live	Routing service wired	E2E routing test	Bug fixes
Negha	FAQ schema + trigger rules	FAQ template design	/ai/generate-faq	/ai/generate-tags + /ai/review-faq	Trigger integration	Auto-approval flow	Bug fixes
Riddhima	State machine doc	Feedback schema with backend	Acceptance criteria write	Approval workflow	Feedback endpoint wired	Demo scenario	Demo rehearsal
Gunjan	Architecture V1	API contract template	Endpoint docs (search)	Endpoint docs (classify, moderate)	Endpoint docs (FAQ gen)	Architecture V2 + presentation	Final slide deck
Rishi	Dataset: 20 FAQs + 15 queries	Dataset: complete + metrics defined	Eval: search endpoint	Eval: classify + moderate	Eval: FAQ gen	Full E2E eval	Final report

07Success Criteria & Evaluation Targets

Metric	Target
Semantic Search Precision@3	≥ 75% (top-3 results contain the correct FAQ in 3/4 test cases)
Semantic Search MRR	≥ 0.70 (correct FAQ ranked first on average)
Query Classification Accuracy	≥ 85% (17/20 labelled queries correctly classified)
Moderation False Positive Rate	≤ 10% (safe queries rejected incorrectly)
FAQ Generation Coherence	≥ 80% of generated FAQs pass quality review auto-approval (score ≥ 0.70)
Tag Relevance	≥ 90% of generated tags rated relevant by human reviewer
End-to-End Happy Path	Query → FAQ served in < 2 seconds (excluding MiniMax latency)
System Availability	No AI endpoint returns 5XX during demo scenario

08Dependencies, Risks & Mitigations

AI Team Dependencies on Other Teams

Need	From
FAQ CRUD API (GET, POST, PUT /faqs)	Backend (Lakshit/Remy/Madhav)
Query API (POST /queries)	Backend
Answer API (GET /answers?queryId=)	Backend
Feedback API (POST /feedback)	Backend
FAQ Search Page (wired to /ai/suggest)	Frontend (Vidhi/Vamsi)
Raise Query Page (wired to /ai/search)	Frontend
MongoDB vector index setup	Database (Damanpreet)

Risk Register

Risk	Mitigation
MiniMax-2 latency too high for live suggestions	Add caching layer (Redis or in-memory LRU) on suggestion endpoint; pre-compute FAQ embeddings at FAQ creation time
MiniMax-2 does not support embedding generation	Gurnoor evaluates this in Day 1 benchmark — fallback: use OpenAI text-embedding-ada-002 or sentence-transformers via local Node.js
Classification accuracy below 85%	Vishal adds more few-shot examples to prompt; Rishi flags which categories fail most
FAQ generation output not JSON-parseable	Samarpit adds retry with stricter JSON-mode instruction; adds validation layer before returning to Negha
Backend APIs not ready by Day 3	Gurnoor builds mock data fixtures for AI team to test against independently
Team member falls behind	Riddhima reviews progress at end of each day; Samarpit's functions must be complete by Day 4 to unblock Vishal and Negha

09

Explicitly Deferred — Future Scope

The following features are NOT planned for MVP. They are documented here so scope creep is visibly blocked.

Out of Scope for This Sprint

- Meeting transcription and speaker identification
- Trend detection across queries over time
- Recommendation engine (suggest related FAQs after reading one)
- Knowledge graph connecting FAQ topics
- Auto-learning and fine-tuning on feedback signals

- Multi-agent architecture for complex query resolution
- Advanced RAG frameworks (LlamaIndex, LangChain)
- Analytics dashboard for FAQ usage and search patterns